Even if the JDBC driver does not support Master-Slave routing, the application can do it using multiple `DataSource` instances. This design cannot rely on the read-only status of the underlying `Connection` since the routing must take place before a database connection is fetched.

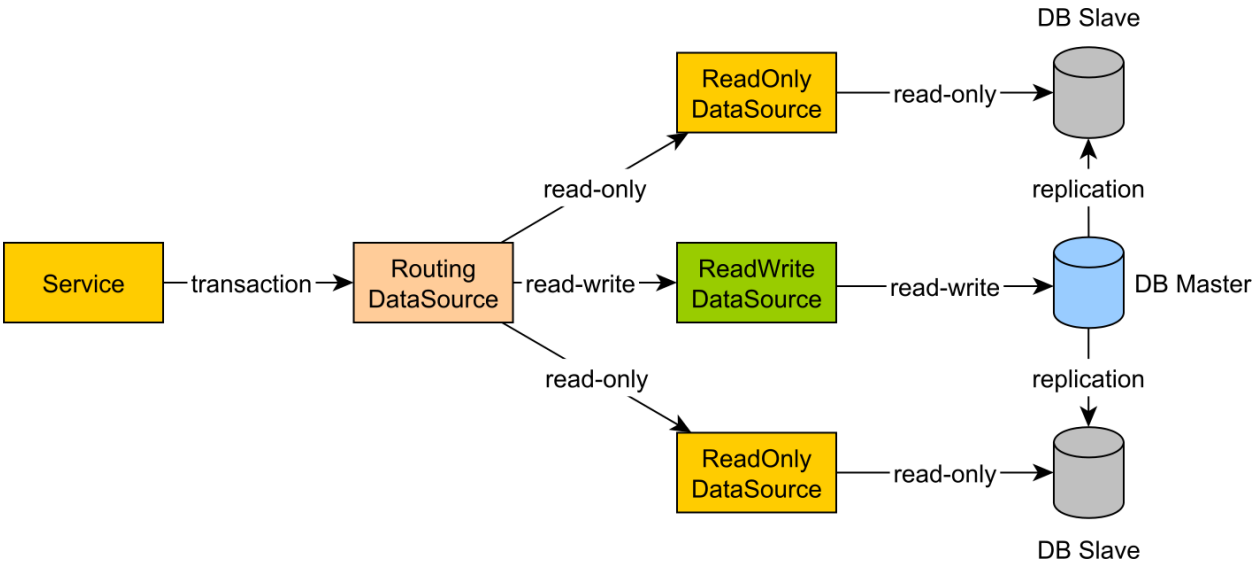

Figure 6.11: Transaction routing

If the transaction manager supports declarative read-only transactions, the routing decision can be taken based on the current transaction read-only preference. Otherwise, the routing must be done manually in each service layer component, and so a read-only transaction uses a read-only `DataSource` or a read-only JPA Persistence Context.

6.6 Transaction boundaries

Every database statement executes in the context of a database transaction, even if the client does not explicitly set transaction boundaries. While there might be single statement transactions (usually a read-only query), when the unit of work consists of multiple SQL statements, the database should wrap them all in a single unit of work.

By default, every `Connection` starts in *auto-commit* mode, each statement being executed in a separate transaction. Unfortunately, it does not work for multi-statement transactions as it moves atomicity boundaries from the logical unit of work to each individual statement.

Auto-commit should be avoided as much as possible, and, even for single statement transactions, it is good practice to mark the transaction boundaries explicitly.

In the following example, a sum of money is transferred between two bank accounts. The balance must always be consistent, so if an account gets debited, the other one must always be credited with the same amount of money.

```
try(Connection connection = dataSource.getConnection();
    PreparedStatement transferStatement = connection.prepareStatement(
        "UPDATE account SET balance = ? WHERE id = ?"
    )) {
    transferStatement.setLong(1, Math.negateExact(cents));
    transferStatement.setLong(2, fromAccountId);
    transferStatement.executeUpdate();

    transferStatement.setLong(1, cents);
    transferStatement.setLong(2, toAccountId);
    transferStatement.executeUpdate();
}
```

Because of the auto-commit mode, if the second statement failed, only those particular changes could be rolled back, the first statement being already committed cannot be reverted anymore.

The default auto-commit mode must be disabled and the transaction has to be managed manually. The transaction is committed if every statement runs successfully and a rollback is triggered on a failure basis. With this in mind, the previous example should be rewritten as follows:

```
try(Connection connection = dataSource.getConnection()) {
    connection.setAutoCommit(false);
    try(PreparedStatement transferStatement = connection.prepareStatement(
        "UPDATE account SET balance = ? WHERE id = ?"
    )) {
        transferStatement.setLong(1, Math.negateExact(cents));
        transferStatement.setLong(2, fromAccountId);
        transferStatement.executeUpdate();

        transferStatement.setLong(1, cents);
        transferStatement.setLong(2, toAccountId);
        transferStatement.executeUpdate();

        connection.commit();
    } catch (SQLException e) {
        connection.rollback();
        throw e;
    }
}
```

The astute reader might notice that the previous example breaks the *Single responsibility principle* since the Data Access Object (DAO) method mixes both transaction management and data access logic. Transaction management is a cross-cutting concern, making it a good candidate for being moved to a separate common library. This way, the transaction management logic resides in one place, and a lot of duplicated code can be removed from the DAO methods. One way to extract the transaction management logic is to use the *Template method pattern*:

```
public void transact(Consumer<Connection> callback) {
    Connection connection = null;
    try {
        connection = dataSource.getConnection();
        callback.accept(connection);
        connection.commit();
    } catch (Exception e) {
        if (connection != null) {
            try {
                connection.rollback();
            } catch (SQLException ex) {
                throw new DataAccessException(e);
            }
        }
        throw (e instanceof DataAccessException ?
            (DataAccessException) e : new DataAccessException(e));
    } finally {
        if(connection != null) {
            try {
                connection.close();
            } catch (SQLException e) {
                throw new DataAccessException(e);
            }
        }
    }
}
```

Transactions should never be abandoned on failure, and it is mandatory to initiate a transaction rollback (to allow the database to revert any uncommitted changes and release any lock as soon as possible).

With this utility in hand, the previous example can be simplified to:

```
transact((Connection connection) -> {
    try(PreparedStatement transferStatement = connection.prepareStatement(
        "UPDATE account SET balance = ? WHERE id = ?"
    )) {
        transferStatement.setLong(1, Math.negateExact(cents));
        transferStatement.setLong(2, fromAccountId);
        transferStatement.executeUpdate();

        transferStatement.setLong(1, cents);
        transferStatement.setLong(2, toAccountId);
        transferStatement.executeUpdate();
    } catch (SQLException e) {
        throw new DataAccessException(e);
    }
});
```

Although better than the first code snippet, separating data access logic and transaction management is not sufficient.

The transaction boundaries are still rigid, and, to include multiple data access method in a single database transaction, the `Connection` object has to be carried out as a parameter to every single DAO method.

Declarative transactions can better address this issue by breaking the strong coupling between the data access logic and the transaction management code. Transaction boundaries are marked with metadata (e.g. annotations) and a separate transaction manager abstraction is in charge of coordinating transaction logic.

Java EE and JTA

Declarative transactions become a necessity for distributed transactions. When Java EE (Enterprise Edition) first emerged, application servers hosted both web applications and middleware integration services, which meant that the Java EE container needed to coordinate multiple `DataSource(s)` or even JMS (Java Messaging) queues.

Following the X/Open XA architecture, JTA (Java Transaction API) powers the Java EE distributed transactions requirements.

6.6.1 Distributed transactions

The difference between local and global transactions is that the former uses a single resource manager, while the latter operates on multiple heterogeneous resource managers. The ACID guarantees are still enforced on each individual resource, but a global transaction manager is mandatory to orchestrate the distributed transaction outcome.

All transactional resource adapters are registered by the global transaction manager, which decides when a resource is allowed to commit or rollback. The Java EE managed resources become accessible through JNDI (Java Naming and Directory Interface) or CDI (Contexts and Dependency Injection).

Spring provides a transaction management abstraction layer which can be configured to either use local transactions (JDBC or RESOURCE_LOCAL[8] JPA) or global transactions through a stand-alone JTA transaction manager. The dependency injection mechanism auto-wires managed resources into Spring beans.

6.6.1.1 Two-phase commit

JTA makes use of the two-phase commit (2PC) protocol to coordinate the atomic resource commitment in two steps: a *prepare* and a *commit* phase.

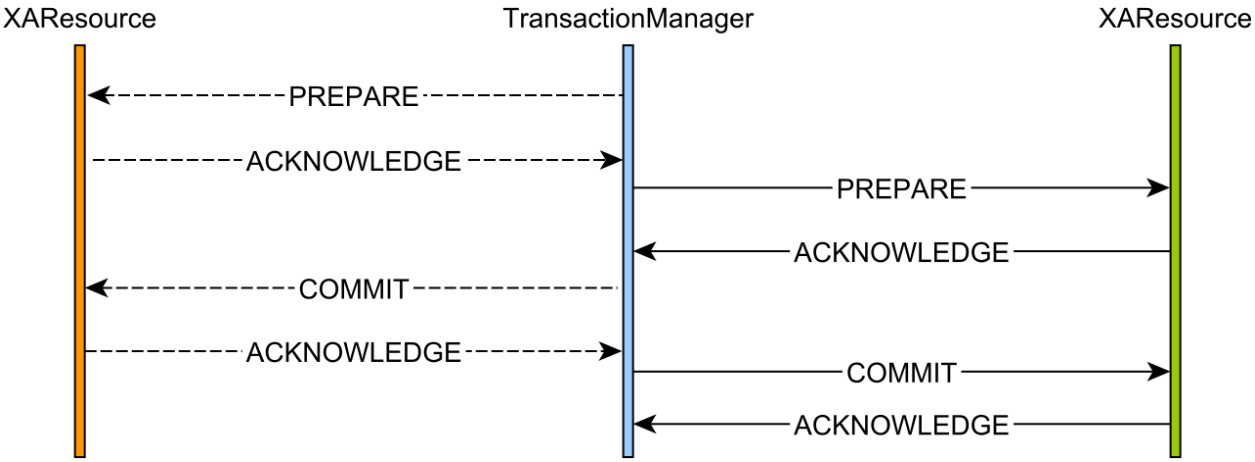

Figure 6.12: Two-phase commit protocol

In the former phase, a resource manager takes all the necessary actions to prepare the transaction for the upcoming commit. Only if all resource managers successfully acknowledge the preparation step, the transaction manager proceeds with the commit phase. If one resource does not acknowledge the prepare phase, the transaction manager will proceed to roll back all current participants.

If all resource managers acknowledge the commit phase, the global transaction will end successfully.

[8] http://docs.oracle.com/javaee/7/api/javax/persistence/spi/PersistenceUnitTransactionType.html#RESOURCE_LOCAL

If one resource fails to commit (or times out), the transaction manager will have to retry this operation in a background thread until it either succeeds or reports the incident for manual intervention.

The one-phase commit (1PC) optimization

Because Java EE uses JTA transactions exclusively, the extra coordination overhead of the additional database roundtrip may hurt performance in a high-throughput application environment. When a transaction enlists only one resource adapter (designating a single resource manager), the transaction manager can skip the prepare phase, and either execute the commit or the rollback phase. With this optimization, the distributed transaction behaves similarly to how JDBC `Connection(s)` manage local transactions.

The `XAResource.commit(Xid xid, boolean onePhase`[a] method takes a `boolean` flag, which the transaction manager sets to *true* to hint the associated resource adapter to initiate the 1PC optimization.

[a] https://docs.oracle.com/javaee/7/api/javax/transaction/xa/XAResource.html#commit-javax.transaction.xa.Xid-boolean-

6.6.2 Declarative transactions

Transaction boundaries are usually associated with a Service layer, which uses one or more DAO to fulfill the business logic. The transaction *propagates* from one component to the other within the service-layer transaction boundaries.

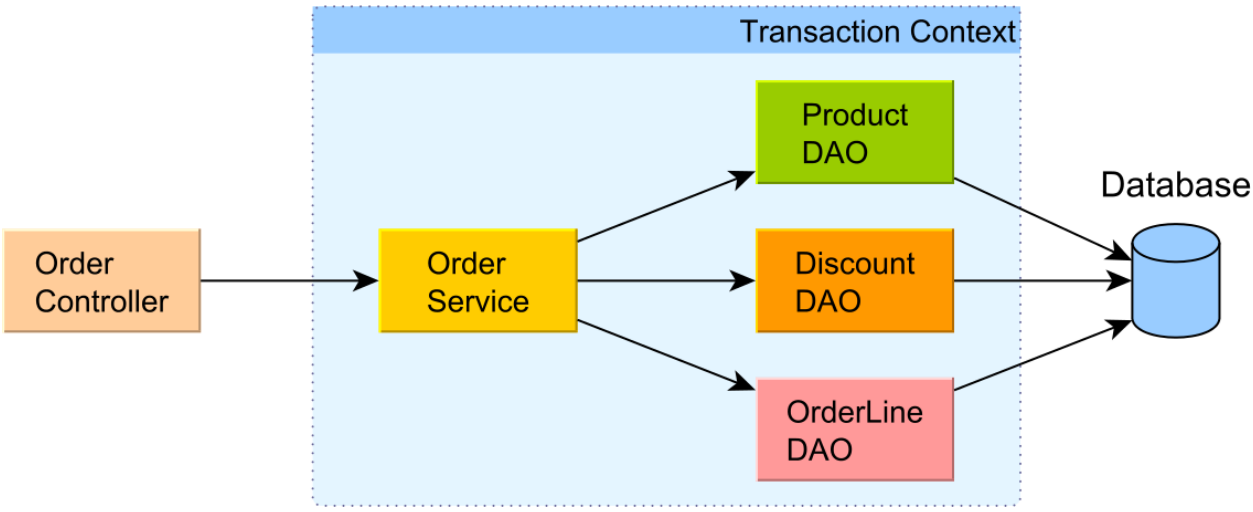

Figure 6.13: Transaction boundaries

The declarative transaction model is supported by both Java EE and Spring. Transaction boundaries are controlled through similar propagation strategies, which define how boundaries are inherited or disrupted at the borderline between the outermost component (in the current call stack) and the current one (waiting to be invoked).

Propagation

To configure the transaction propagation strategy for EJB components, Java EE defines the `@TransactionAttribute`[a] annotation. Since Java EE 7, even non-EJB components can now be enrolled in a transactional context if they are augmented with the `@Transactional`[b] annotation.

In Spring, transaction propagation (like any other transaction properties) is configurable via the `@Transactional`[c] annotation.

[a] http://docs.oracle.com/javaee/7/api/javax/ejb/TransactionAttribute.html
[b] http://docs.oracle.com/javaee/7/api/javax/transaction/Transactional.html
[c] https://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/transaction/annotation/Transactional.html#propagation--

Table 6.6: Transaction propagation strategies

Propagation	Java EE	Spring	Description
REQUIRED	Yes	Yes	This is the default propagation strategy, and it only starts a transaction unless the current thread is not already associated with a transaction context
REQUIRES_NEW	Yes	Yes	Any currently running transaction context is suspended and replaced by a new transaction
SUPPORTS	Yes	Yes	If the current thread already runs inside a transaction, this method will use it. Otherwise, it executes outside of a transaction context
NOT_SUPPORTED	Yes	Yes	Any currently running transaction context is suspended, and the current method is run outside of a transaction context
MANDATORY	Yes	Yes	The current method runs only if the current thread is already associated with a transaction context
NESTED	No	Yes	The current method is executed within a nested transaction if the current thread is already associated with a transaction. Otherwise, a new transaction is started.
NEVER	Yes	Yes	The current method must always run outside of a transaction context, and, if the current thread is associated with a transaction, an exception will be thrown.

Declarative exception handling

Since the transaction logic wraps around the underlying service and data access logic call chain, the exception handling must also be configured declaratively. By default, both Java EE and Spring roll back on system exceptions (any `RuntimeException`) and commit on application exceptions (checked exceptions).

In Java EE, the rollback policy can be customized using the `@ApplicationException`[a] annotation.

Spring allows each transaction to customize the rolling back policy[b] by listing the exception types triggering a transaction failure.

[a] http://docs.oracle.com/javaee/7/api/javax/ejb/ApplicationException.html
[b] https://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/transaction/annotation/Transactional.html#rollbackFor--

Declarative read-only transactions

Java EE does not support read-only transactions to be marked declaratively.

Spring offers the transactional read-only attribute[a], which can propagate to the underlying JPA provider (to optimize the `EntityManager` flushing mechanism) and to the current associated JDBC `Connection`.

[a] https://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/transaction/annotation/Transactional.html#readOnly--

Declarative isolation levels

The Java EE does not offer support for configurable isolation levels, so it is up to the underlying `DataSource` to define it for all database connections.

Spring supports transaction-level isolation levels[a] when using the `JpaTransactionManager`[b]. For JTA transactions, the `JtaTransactionManager`[c] follows the Java EE standard and disallows overriding the default isolation level. As a workaround, the Spring framework provides extension points, so the application developer can customize the default behavior and implement a mechanism to set isolation levels on a transaction basis.

[a] https://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/transaction/annotation/Transactional.html#isolation--
[b] https://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/orm/jpa/JpaTransactionManager.html
[c] https://docs.spring.io/spring/docs/current/javadoc-api/org/springframework/transaction/jta/JtaTransactionManager.html

6.7 Application-level transactions

So far, the book focused on database transactions to enforce ACID properties. However, from the application perspective, a business workflow might span over multiple physical database transactions, in which case the database ACID guarantees are not sufficient anymore. A logical transaction may be composed of multiple web requests, including user think time, for which reason it can be visualized as a *long conversation*.

In the following example, *Alice* and a background process are concurrently modifying the same database record.

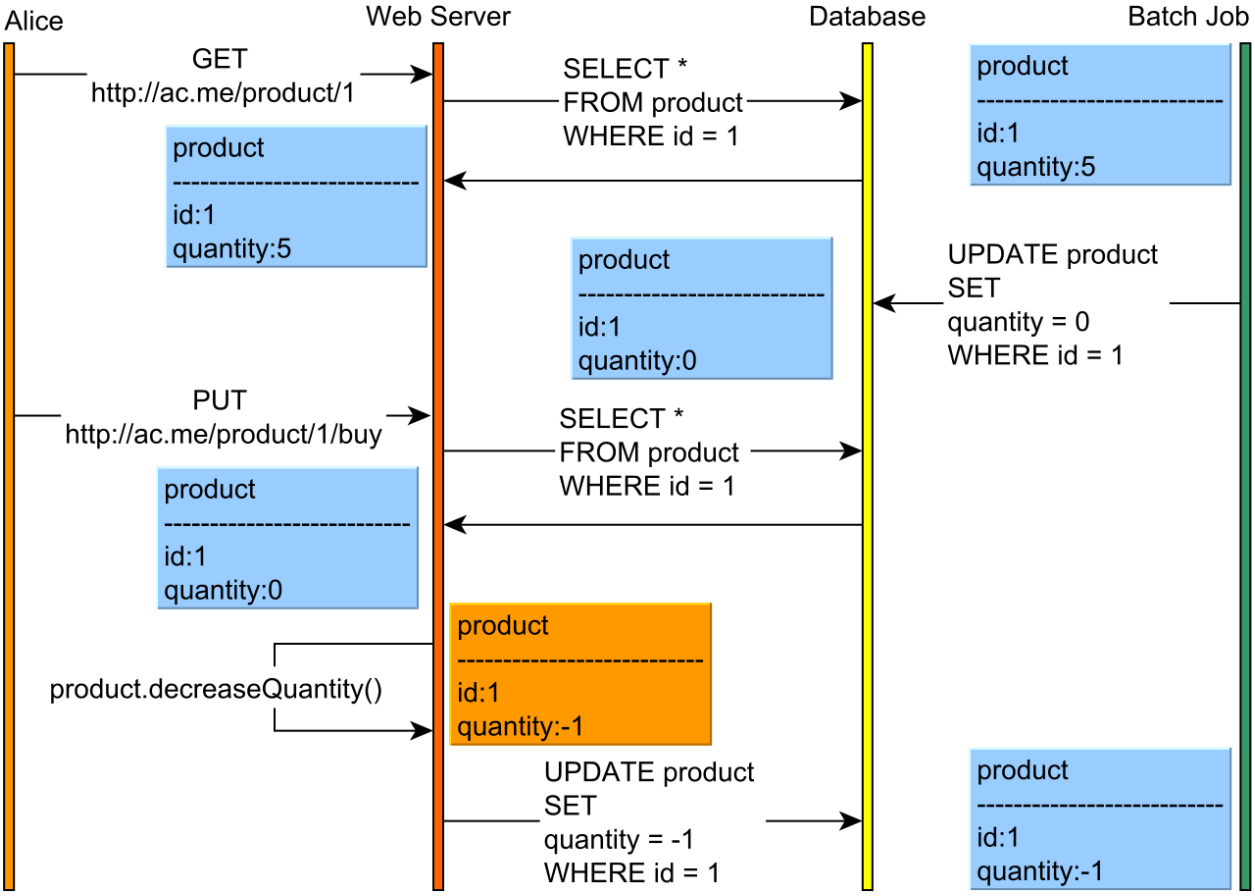

Figure 6.14: Stateless conversation losing updates

Because *Alice* logical transaction encloses two separate web requests, each one associated with a separate database transaction, without an additional concurrency control mechanism, even the strongest isolation level cannot prevent the lost update phenomena.

Spanning a database transaction over multiple web requests is prohibitive since locks would be held during user think time, therefore hurting scalability. Even with MVCC, the cost of maintaining previous versions (that can lead to a large version graph) can escalate and affect both performance and concurrency. Application-level transactions require application-level concurrency control mechanisms.

HTTP is stateless by nature and, for very good reasons, stateless applications are easier to scale than stateful ones. However, application-level transactions cannot be stateless, as otherwise newer requests would not continue from where the previous request was left. Preserving state across multiple web requests allows building a conversational context, providing application-level repeatable reads guarantees.

In the next diagram, *Alice* uses a stateful conversational context, but, in the absence of a record versioning system, it is still possible to lose updates.

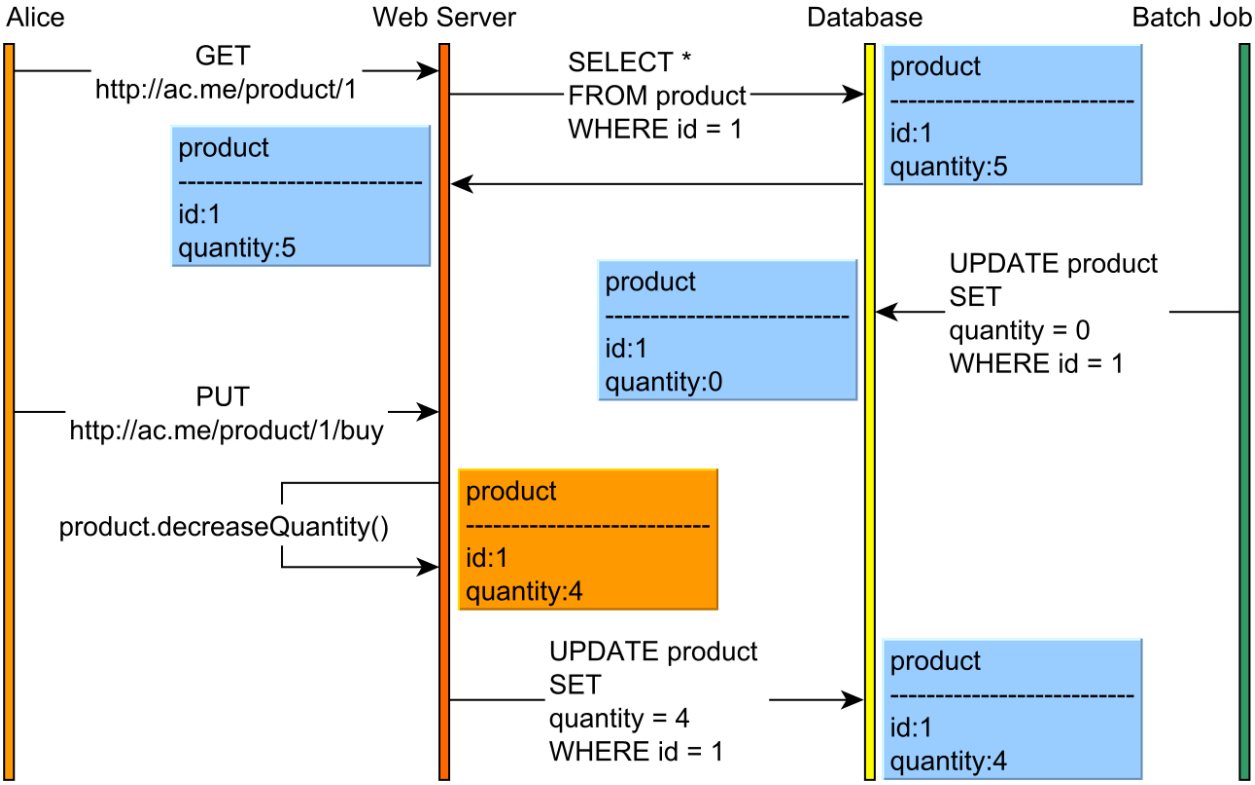

Figure 6.15: Stateful conversation losing updates

Without *Alice* to notice, the batch process resets the product quantity. Thinking the product version has not changed, *Alice* attempts to purchase one item which decreases the previous product quantity by one. In the end, *Alice* has simply overwritten the batch processor modification, and data integrity has been compromised.

So the application-level repeatable reads are not self-sufficient (this argument is true for database isolation levels as well). To prevent lost updates, a concurrency control mechanism becomes mandatory.

6.7.1 Pessimistic and optimistic locking

Isolation levels entail implicit locking, whether it involves physical locks (like 2PL) or data anomaly detection (MVCC). To coordinate state changes, application-level concurrency control makes use of explicit locking, which comes in two flavors: pessimistic and optimistic locking.

6.7.1.1 Pessimistic locking

As previously explained, most database systems already offer the possibility of manually requesting shared or exclusive locks. This concurrency control is said to be *pessimistic* because it assumes that conflicts are bound to happen, and so they must be prevented accordingly.

> As locks can be released in a timely fashion, exclusive locking is appropriate during the last database transaction of a given long conversation. This way, the application can guarantee that, once locks are acquired, no other transaction can interfere with the currently locked resources.
>
> Acquiring locks on critical records can prevent non-repeatable reads, lost updates, as well as read and write skew phenomena.

6.7.1.2 Optimistic locking

Undoubtedly a misnomer (albeit rather widespread), optimistic locking does not incur any locking at all. A much better name would be optimistic concurrency control since it uses a totally different approach to managing conflicts than pessimistic locking.

MVCC is an optimistic concurrency control strategy since it assumes that contention is unlikely to happen, and so it does not rely on locking for controlling access to shared resources. The optimistic concurrency mechanisms detect anomalies and resort to aborting transactions whose invariants no longer hold.

While the database knows exactly which row versions have been issued in a given time interval, the application is left to maintaining a *happens-before* event ordering. Each database row must have an associated version, which is locally incremented by the logical transaction. Every modifying SQL statement (update or delete) uses the previously loaded version as an assumption that the row has not been changed in the meanwhile.

Because even the lowest isolation level can prevent write-write conflicts, only one transaction is allowed to update a row version at any given time. Since the database already offers monotonic updates, the row versions can also be incremented monotonically, and the application can detect when an updating record has become stale.

The optimistic locking concurrency algorithm looks like this:

- When a client reads a particular row, its version comes along with the other fields.
- Upon updating a row, the client filters the current record by the version it has previously loaded.

```
UPDATE product
SET (quantity, version) = (4, 2)
WHERE id = 1 AND version = 1
```

- If the statement update count is zero, the version was incremented in the meanwhile, and the current transaction now operates on a stale record version.

The previous example can be adjusted to take advantage of this optimistic concurrency control mechanism. This time, the product is versioned, and both the web application and the batch processor data access logic are using the row versions to coordinate the *happens-before* update ordering.

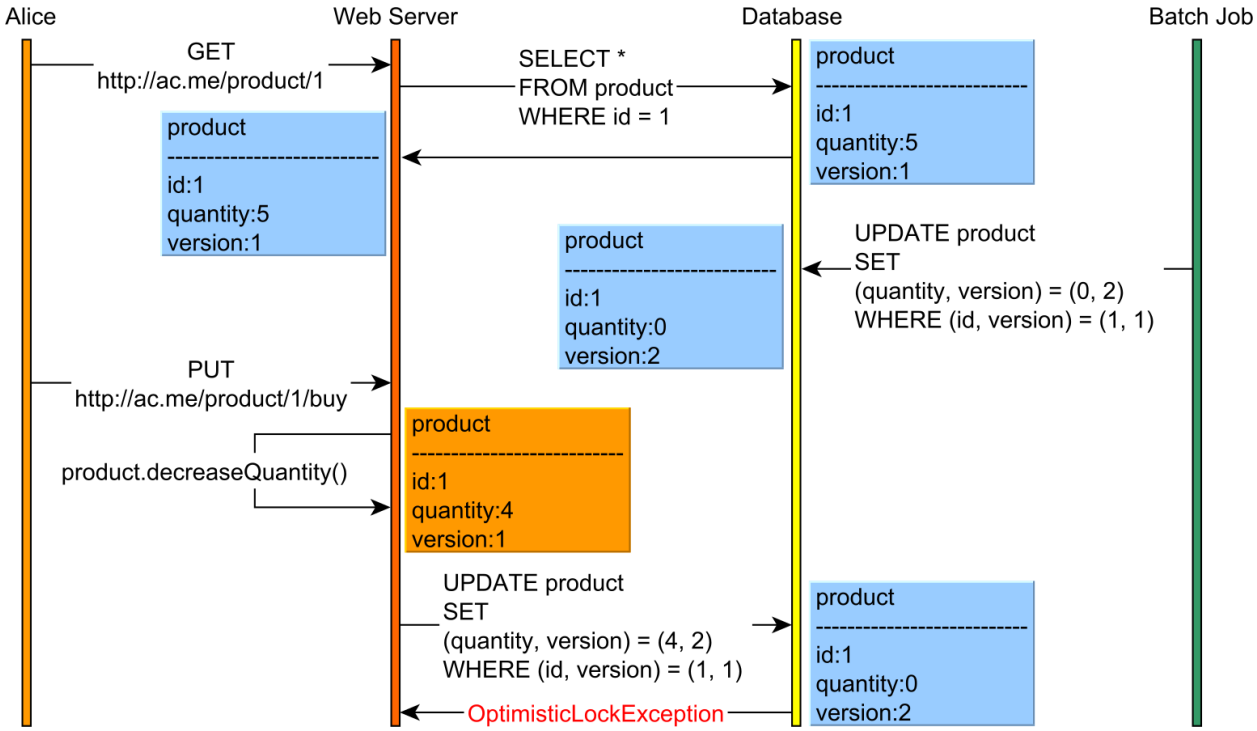

Figure 6.16: Stateful conversation preventing lost updates

Both *Alice* and the batch processor try to increment the product version optimistically. The batch processor can successfully update the product quantity since the SQL statement filtering criteria matches the actual database record version.

When *Alice* tries to update the product, the database returns a zero update count, and, this way, she is notified about the concurrent update that happened in the meanwhile.

The lost update can be prevented because the application can abort the current transaction when being notified of the stale record versions.

II JPA and Hibernate

7. Why JPA and Hibernate matter

Although JDBC does a very good job of exposing a common API that hides the database vendor-specific communication protocol, it suffers from the following shortcomings:

- The API is undoubtedly verbose, even for trivial tasks.
- Batching is not transparent from the data access layer perspective, requiring a specific API than its non-batched statement counterpart.
- Lack of built-in support for explicit locking and optimistic concurrency control.
- For local transactions, the data access is tangled with transaction management semantics.
- Fetching joined relations requires additional processing to transform the `ResultSet` into Domain Models or DTO (Data Transfer Object) graphs.

Although the primary goal of an ORM (Object-Relational Mapping) tool is to automatically translate object state transitions into SQL statements, this chapter aims to demonstrate that Hibernate can address all the aforementioned JDBC shortcomings.

Java Persistence history

The EJB 1.1 release offered a higher-level persistence abstraction through scalable enterprise components, known as Entity Beans. Although the design looked good on paper, in reality, the heavyweight RMI-based implementation proved to be disastrous from a performance perspective. Neither the EJB 2.0 support for *local interfaces* could revive the Entity Beans popularity, and, due to high-complexity and vendor-specific implementation details, most projects chose JDBC instead.

Hibernate was born out of all the frustration of using the Entity Bean developing model. As an open-source project, Hibernate managed to gain a lot of popularity, and so it soon became the *de facto* Java persistence framework.

In response to all the criticism associated with Entity Bean persistence, the Java Community Process advanced a lightweight POJO-based approach, and the JDO specification was born. Although JDO is data source agnostic, being capable of operating with both relation databases as well as NoSQL or even flat files, it never managed to hit mainstream popularity. For this reason, the Java Community Process decided that EJB3 would be based on a new specification, inspired by Hibernate and TopLink, and JPA (Java Persistence API) became the standard Java persistence technology.

The morale of this is that persistence is a very complex topic, and it demands a great deal of knowledge of both the database and the data access usage patterns.

7.1 The impedance mismatch

When a relational database is manipulated through an object-oriented program, the two different data representations start conflicting.

In a relational database, data is stored in tables, and the relational algebra defines how data associations are formed. On the other hand, an object-oriented programming (OOP) language allows objects to have both state and behavior, and bidirectional associations are permitted.

The burden of converging these two distinct approaches has generated much tension, and it has been haunting enterprise systems for a very long time.

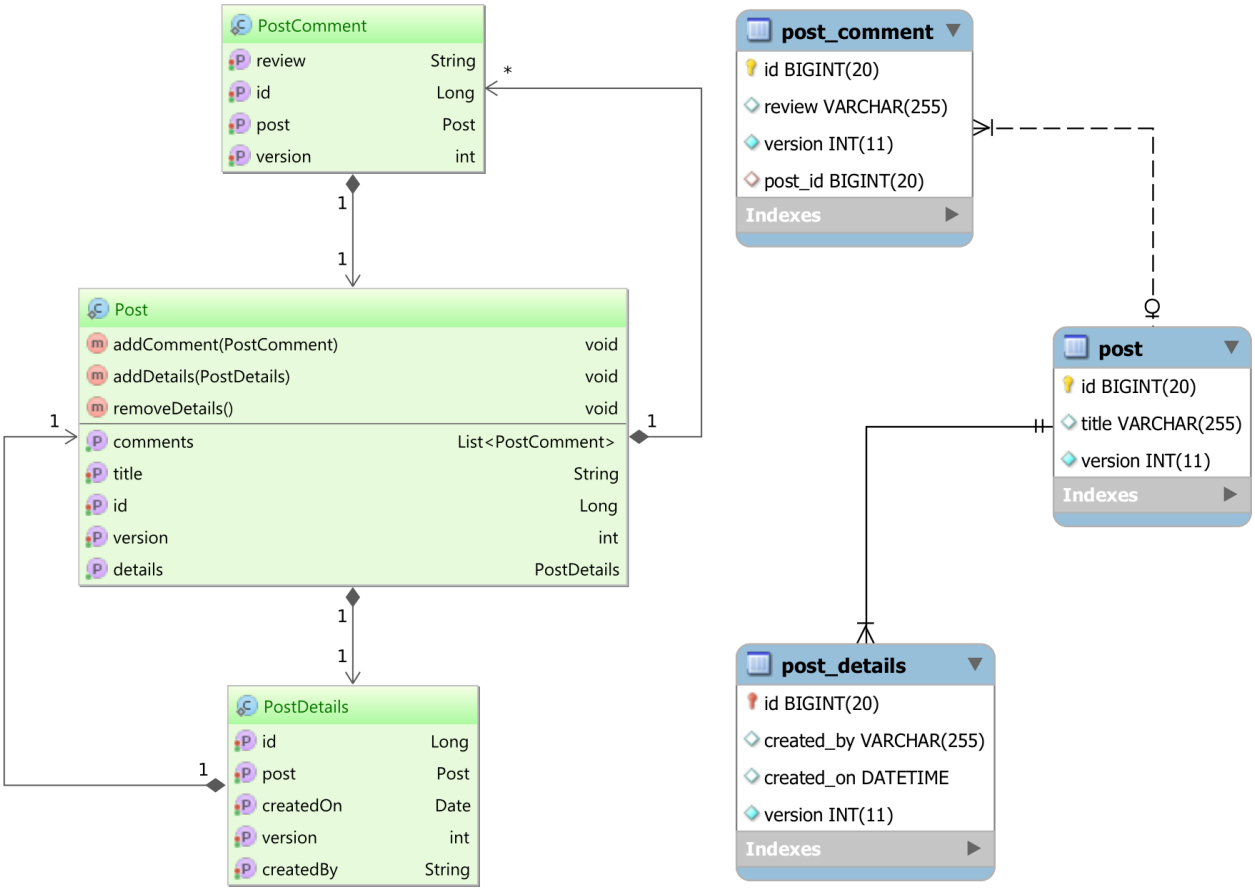

Figure 7.1: Object/Relational Mapping

The above diagram portrays the two different schemas that the data access layer needs to correlate. While the database schema is driven by the SQL standard specification, the Domain Model comes with an object-oriented schema representation as well.

The Domain Model encapsulates the business logic specifications and captures both data structures and the behavior that governs business requirements. OOP facilitates Domain Modeling, and many modern enterprise systems are implemented on top of an object-oriented programming language.

Because the underlying data resides in a relational database, the Domain Model must be adapted to the database schema and the SQL-driven communication protocol. The ORM design pattern helps to bridge these two different data representations and close the technological gap between them. Every database row is associated with a Domain Model object (*Entity* in JPA terminology), and so the ORM tool can translate the entity state transitions into DML statements.

From an application development point of view, this is very convenient since it is much easier to manipulate Domain Model relationships rather than visualizing the business logic through its underlying SQL statements.

7.2 JPA vs. Hibernate

JPA is only a specification. It describes the interfaces that the client operates with and the standard object-relational mapping metadata (Java annotations or XML descriptors). Beyond the API definition, JPA also explains (although not exhaustively) how these specifications are ought to be implemented by the JPA providers. JPA evolves with the Java EE platform itself (Java EE 6 featuring JPA 2.0 and Java EE 7 introducing JPA 2.1).

Hibernate was already a full-featured Java ORM implementation by the time the JPA specification was released for the first time. Although it implements the JPA specification, Hibernate retains its native API for both backward compatibility and to accommodate non-standard features.

Even if it is best to adhere to the JPA standard, in reality, many JPA providers offer additional features targeting a high-performance data access layer requirements. For this purpose, Hibernate comes with the following non-JPA compliant features:

- extended identifier generators (hi/lo, pooled, pooled-lo)
- transparent prepared statement batching
- customizable CRUD (`@SQLInsert`, `@SQLUpdate`, `@SQLDelete`) statements
- static/dynamic entity/collection filters (e.g. `@FilterDef`, `@Filter`, `@Where`)
- mapping attributes to SQL fragments (e.g. `@Formula`)
- immutable entities (e.g. `@Immutable`)
- more flush modes (e.g. `FlushMode.MANUAL`, `FlushMode.ALWAYS`)
- querying the second-level cache by the natural key of a given entity
- entity-level cache concurrency strategies
  (e.g. `Cache(usage = CacheConcurrencyStrategy.READ_WRITE)`)
- versioned bulk updates through HQL
- exclude fields from optimistic locking check (e.g. `@OptimisticLock(excluded = true)`)
- versionless optimistic locking (e.g. `OptimisticLockType.ALL`, `OptimisticLockType.DIRTY`)
- support for skipping (without waiting) pessimistic lock requests
- support for Java 8 Date and Time and `stream()`
- support for multitenancy

If JPA is the interface, Hibernate is one implementation and implementation details always matter from a performance perspective.

The JPA implementation details leak and ignoring them might hinder application performance or even lead to data inconsistency issues. As an example, the following JPA attributes have a peculiar behavior, which can surprise someone who is familiar with the JPA specification only:

- The FlushModeType.AUTO[1] does not trigger a flush for native SQL queries like it does for JPQL or Criteria API.
- The FetchType.EAGER[2] might choose a SQL join or a secondary select whether the entity is fetched directly from the `EntityManager` or through a JPQL (Java Persistence Query Language) or a Criteria API query.

That is why this book is focused on how Hibernate manages to implement both the JPA specification and its non-standard native features (that are relevant from an efficiency perspective).

Portability concerns

Like other non-functional requirements, portability is a feature, and there is still a widespread fear of embracing database-specific or framework-specific features. In reality, it is more common to encounter enterprise applications facing data access performance issues than having to migrate from one technology to the other (be it a relational database or a JPA provider).

The lowest common denominator of many RDBMS is a superset of the SQL-92 standard (although not entirely supported either). SQL-99 supports Common Table Expressions, but MySQL 5.7 does not. Although SQL-2003 defines the `MERGE` operator, PostgreSQL 9.5 favored the `UPSERT` operation instead. By adhering to a SQL-92 syntax, one could achieve a higher degree of database portability, but the price of giving up database-specific features can take a toll on application performance. Portability can be addressed either by subtracting non-common features or through specialization. By offering different implementations, for each supported database system (like the jOOQ framework does), portability can still be achieved.

The same argument is valid for JPA providers too. By layering the application, it is already much easier to swap JPA providers, if there is even a compelling reason for switching one mature JPA implementation with another.

[1] https://docs.oracle.com/javaee/7/api/javax/persistence/FlushModeType.html
[2] https://docs.oracle.com/javaee/7/api/javax/persistence/FetchType.html#EAGER

7.3 Schema ownership

Because of data representation duality, there has been a rivalry between taking ownership of the underlying schema. Although theoretically, both the database and the Domain Model could drive the schema evolution, for practical reasons, the schema belongs to the database.

An enterprise system might be too large to fit into a single application, so it is not uncommon to split in into multiple subsystems, each one serving a specific goal. As an example, there can be front-end web applications, integration web services, email schedulers, full-text search engines and back-end batch processors that need to load data into the system. All these subsystems need to use the underlying database, whether it is for displaying content to the users or dumping data into the system.

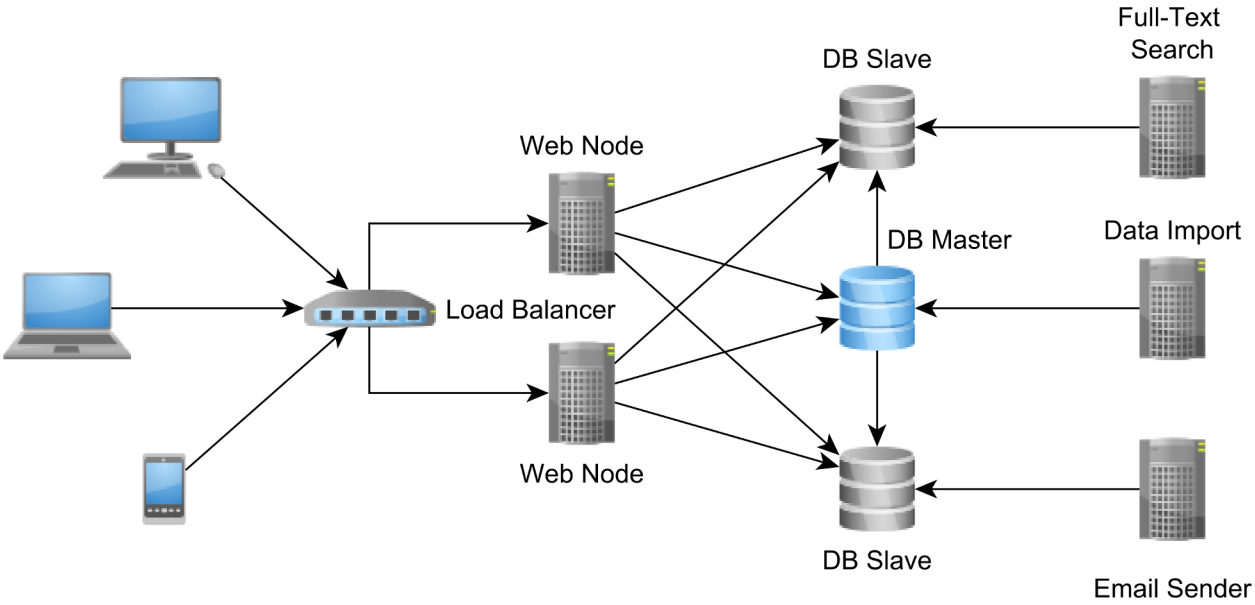

Figure 7.2: Database-centric integration

Although it might not fit any enterprise system, having the database as a central integration point can still be a choice for many reasonable size enterprise systems.

The relational database concurrency models offer strong consistency guarantees, therefore having a significant advantage to application development. If the integration point does not provide transactional semantics, it will be much more difficult to implement a distributed concurrency control mechanism.

Most database systems already offer support for various replication topologies, which can provide more capacity for accommodating an increase in the incoming request traffic. Even if the demand for more data continues to grow, the hardware is always getting better and better (and cheaper too), and database vendors keep on improving their engines to cope with more data.

For these reasons, having the database as an integration point is still a relevant enterprise system design consideration.

The distributed commit log

For very large enterprise systems, where data is split among different providers (relational database systems, caches, Hadoop, Spark), it is no longer possible to rely on the relational database to integrate all disparate subsystems.

In this case, Apache Kafka[a] offers a fault-tolerant and scalable append-only log structure, which every participating subsystem can read and write concurrently.

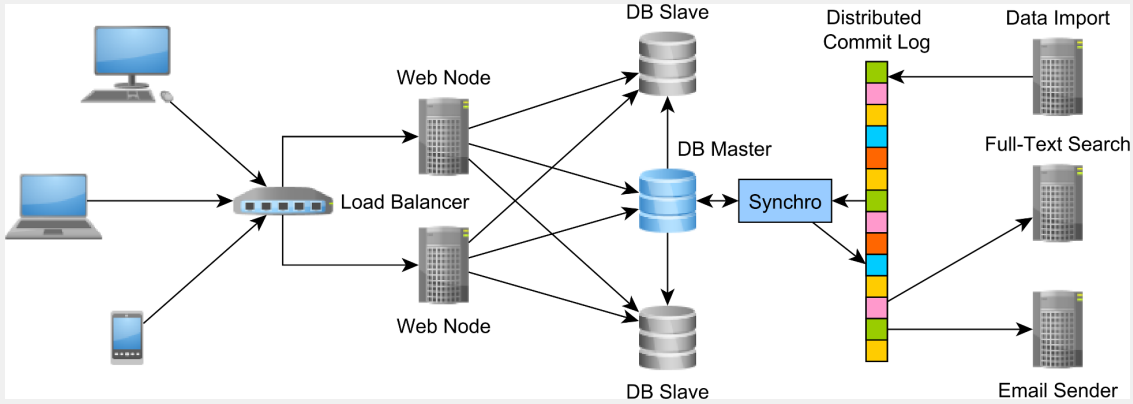

Figure 7.3: Distributed commit log integration

The commit log becomes the integration point, each distributed node individually traversing it and maintaining client-specific pointers in the sequential log structure. This design resembles a database replication mechanism, and so it offers durability (the log is persisted on disk), write performance (append-only logs do not require random access) and read performance (concurrent reads do not require blocking) as well.

[a] http://kafka.apache.org/

No matter what architecture style is chosen, there is still need to correlate the transient Domain Model with the underlying persistent data.

The data schema evolves along the enterprise system itself, and so the two different schema representations must remain congruent at all times.

Even if the data access framework can auto-generate the database schema, the schema must be migrated incrementally, and all changes need to be traceable in the VCS (Version Control System) as well. Along with table structure, indexes and triggers, the database schema is, therefore, accompanying the Domain Model source code itself. A tool like Flywaydb[3] can automate the database schema migration, and the system can be deployed continuously, whether it is a test or a production environment.

[3] http://flywaydb.org/